Listing 8.7 `langchain_token_splitting.py` (relevant new code)

```
loader = UnstructuredHTMLLoader("sample_documents/mother_goose.xhtml")
data = loader.load()
text_splitter = CharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=50, chunk_overlap=10      #1
)

documents = text_splitter.split_documents(data)
documents = [doc for doc in documents][8:94]      #2

db = Chroma.from_documents(documents, OpenAIEmbeddings())

def query_documents(query, top_n=2):
    docs = db.similarity_search(query, top_n)      #3
    return docs
###Output
Created a chunk of size 68,
which is longer than the specified 50
Created a chunk of size 67,
which is longer than the specified 50     #4
Enter a search query (or 'exit' to stop):
                    who kissed the girls and made them cry?
How many top matches do you want to see? 3
Top Matched Documents:
Document 1: GEORGY PORGY

        Georgy Porgy, pudding and pie,
        Kissed the girls and made them cry.
```

#1 Updates to 50 tokens and overlap of 10 tokens
#2 Selects just the documents that contain rhymes
#3 Uses the database's similarity search
#4 Breaks into irregular size chunks because of the whitespace

Run the `langchain_token_splitting.py` code in VS Code (F5). You can use the query we used last time or your own. Notice how the results are significantly better than the previous exercise. However, the results are still suspect because the query uses several similar words in the same order.

A better test would be to try a semantically similar phrase but one that uses different words and check the results. With the code still running, enter a new phrase to query: `Why are the girls crying?` Listing 8.8 shows the results of executing that query. If you run this example yourself and scroll down over the output, you'll see Georgy Porgy appear in either the second or third returned document.